Atlas Reflection

Key Learnings

The main lesson from Atlas was that the quality of an AI application is decided before the first line of code. AI coding tools can scaffold pages, routes, tests, and documentation quickly. That makes implementation cheaper, but it also makes it possible to build the wrong product faster. I found that the most valuable work was early: defining a real problem, choosing a narrow user, comparing alternatives, and writing down what the product should and should not do.

I began with validation instead of features. The seven-step framework helped me turn a broad career-coach idea into a specific workflow: a job seeker has a resume and target role but does not know what evidence matters or what action to take next. Research on early-career employment, changing skills, existing resume tools, and paid alternatives clarified the gap. Atlas should not try to replace job boards, ATS scanners, or mentors. Its role is to turn a role-specific review into a practical, trackable action plan.

The PRD and specification made this decision executable. The PRD set the target user, MVP, success metrics, and boundaries. The specification then defined the inputs, outputs, behavior rules, privacy constraints, architecture, and acceptance criteria. These documents made AI-assisted development more precise: instead of asking an agent to make a generic career app, I could ask it to build a small, testable part of an agreed workflow. Careful brainstorming, planning, and specifications are now a better use of time than trying to write every line manually. Code is cheaper to produce; product judgment is still scarce.

Challenges and Scope Control

The largest challenge was controlling scope within the capstone timeline. It was easy to imagine LinkedIn integration, job-board search, automatic applications, broad career courses, social sharing, and a full game system. Each idea sounded useful, but together they would have prevented a reliable MVP. I kept Atlas centered on one path: sign in, add a resume and target role, review the extracted text, generate a readiness report, complete a quest, and ask a report-specific follow-up question.

Technical scope control mattered as much as product scope. The implementation combined document extraction, structured LLM output, curated RAG guidance, Supabase Auth and Row Level Security, a readiness dashboard, and a deployed document service. I treated privacy and reliability as product requirements: uploaded files and full raw resume text are not stored, private career data is not added to the shared RAG corpus, and the fit score is guidance rather than a hiring prediction. Testing validation, schemas, permissions, document safety, and deployment paths showed me that an AI feature needs more than a model call.

Future Versions

Testing and deployment validated the MVP. The next priority is improving what happens after a report is generated: make each recommended action more specific, timely, and connected to the user's goal and progress.

An adaptive recommendation engine could rank next actions from the user's role, skill gaps, and completed quests. A redesigned campaign UI could make momentum visible through milestones, progress maps, and private rewards tied to real career evidence.

Useful next modules include career-path presets, a portfolio-evidence tracker, job-market research, and opt-in LinkedIn profile insights. These should extend the guidance workflow, not automate applications or promise hiring outcomes.

Overall, Atlas showed me how validation, planning, implementation, testing, and deployment fit together. AI can accelerate delivery, but I remain responsible for deciding what is worth building, keeping the scope realistic, and checking that the final product is safe and useful.